Table of Contents

1. Introduction
    1.1. Scope and Assumptions
2. How To Read This Document
3. Document Health Snapshot
4. Terms and Definitions
5. Functional View
    5.1. What This View Answers
    5.2. Coverage Gaps
    5.3. Recommended Next Evidence Additions
    5.4. Functional Decomposition Diagram
    5.5. Functional Collaboration Diagram
    5.6. Functional Groups (View Scoped)
    5.7. Functional Units (View Scoped)
6. Runtime View
    6.1. What This View Answers
    6.2. Coverage Gaps
    6.3. Recommended Next Evidence Additions
    6.4. Functional Groups (View Scoped)
    6.5. Functional Units (View Scoped)
7. Deployment View
    7.1. What This View Answers
    7.2. Coverage Gaps
    7.3. Recommended Next Evidence Additions
    7.4. Platform Operations Graph
    7.5. Functional Groups (View Scoped)
    7.6. Functional Units (View Scoped)
8. Realization View
    8.1. What This View Answers
    8.2. Coverage Gaps
    8.3. Recommended Next Evidence Additions
    8.4. Code Dependency Graph
    8.5. Functional Groups (View Scoped)
    8.6. Functional Units (View Scoped)
9. Security View
    9.1. What This View Answers
    9.2. Coverage Gaps
    9.3. Recommended Next Evidence Additions
    9.4. Security Logging and Observability
    9.5. Functional Groups (View Scoped)
    9.6. Functional Units (View Scoped)
10. Generated Evidence Appendix
11. Requirement Alignment
    11.1. Cross-Layer Coverage

11.2. Requirement Details
12. Reference Index
12.1. Authored References
12.2. Catalog References
12.3. Inferred Runtime References
12.4. Inferred Code References

---

1. Introduction

This document describes the payments architecture using authored functional design and inferred runtime/code realization. It is intended to help product, platform, security, and implementation engineers reason about one shared model. Functional design is kept stable while realization details are inferred from infrastructure and source artifacts.

1.1. Scope and Assumptions

- Functional Groups and Functional Units are the stable authored design backbone.
- Runtime and code realization are inferred from available IaC, deployment, and source artifacts.
- Inferred coverage is evidence-driven and may be partial when source metadata is missing.
- Requirement alignment and reference indexes are provided for traceability, not as primary narrative flow.

2. How To Read This Document

This document is intentionally split into two layers:

1. Concept layer: functional and view-specific architecture narratives for human understanding.
2. Evidence layer: generated requirement traceability and inferred reference indexes.

Read in this order:

1. Functional View: authored architecture intent and responsibilities.
2. Runtime/Deployment/Realization/Security Views: view-specific realization details.
3. Generated Evidence Appendix: traceability and inferred references for audit/tooling.

3. Document Health Snapshot

View	Units in Scope	Direct Evidence Coverage	Gap Count	Confidence
Functional View (`VIEW-FUNCTIONAL`)	6	6	0	high
Runtime View (`VIEW-RUNTIME`)	4	3	1	medium
Deployment View (`VIEW-DEPLOYMENT`)	6	5	1	high

View	Units in Scope	Direct Evidence Coverage	Gap Count	Confidence
Realization View (`VIEW-CODE-OWNERSHIP`)	4	3	1	medium
Security View (`VIEW-SECURITY`)	6	2	4	low

4. Terms and Definitions

Term	Definition
3DS verification is required (COND-3DS-REQUIRED)	Condition indicating strong customer authentication must be performed. Aliases:3DS required
actor (EM-ACTOR)	A person or operational role that interacts with functional units.
attack vector (EM-ATTACK-VECTOR)	A technical misuse or attack path that targets functional, referenced, or runtime elements.
authored mapping (EM-AUTHORED-MAPPING)	An explicit relationship declared in architecture inputs between authored or referenced elements.
bank gateway endpoint (REF-BANK-GATEWAY-ENDPOINT)	External banking endpoint used for card authorization and decline decisions. Aliases:bank API
bank link is available (STATE-BANK-LINK-UP)	Connectivity state showing bank gateway communication is currently healthy. Aliases:bank link up
bank link is unavailable (EVT-BANK-LINK-UNAVAILABLE)	Event raised when payment engine cannot reach the external bank gateway. Aliases:bank link down
checkout handling (FU-CHECKOUT)	Handles checkout session start and customer-facing payment messaging. Aliases:checkout
checkout is active (STATE-CHECKOUT-ACTIVE)	Processing state where the user is actively submitting or confirming payment. Aliases:checkout active
cluster provisioning (FU-CLUSTER-PROVISIONING)	Provides cluster-level platform setup and environment bootstrap. Aliases:day1 provisioning
code element (EM-CODE-ELEMENT)	An inferred code structure or ownership element discovered from source trees and build metadata.
compromised dependency (AV-COMPROMISED-DEPENDENCY)	Supply-chain compromise in runtime image or code dependency. Aliases:dependency compromise
customer (ACT-CUSTOMER)	End user submitting payment details during checkout. Aliases:buyer

Term	Definition
decline reason (DATA-DECLINE-REASON)	Structured reason code or message explaining why authorization failed. Aliases:decline code
fraud check is enabled (FEAT-FRAUD-CHECK)	Feature flag controlling whether risk scoring is required before authorization. Aliases:fraud check
fraud evaluation (FG-FRAUD)	Functional group for risk scoring and fraud review capabilities. Aliases:fraud detection system, fraud domain
functional group (EM-FUNCTIONAL-GROUP)	A major authored capability area that groups related functional units.
functional unit (EM-FUNCTIONAL-UNIT)	An authored working unit inside a functional group that owns specific behavior.
gitops operations (FU-GITOPS-OPERATIONS)	Operates declarative delivery and release management flows. Aliases:day2 operations
helm platform (REF-HELM-PLATFORM)	Runtime packaging and deployment substrate for workloads. Aliases:helm
inferred mapping (EM-INFERRED-MAPPING)	A discovered relationship that links inferred runtime/code elements upward to authored design.
malicious api request (AV-MALICIOUS-API-REQUEST)	Crafted payload attempting to bypass validation or authorization logic. Aliases:malicious request
normal mode is active (MODE-NORMAL)	Standard operating mode with full external dependency usage enabled. Aliases:normal mode
payment authorization (FU-PAYMENT-AUTHORIZATION)	Coordinates authorization flow and bank gateway interaction. Aliases:auth
payment authorization is requested (EVT-PAYMENT-AUTH-REQUESTED)	Event raised when checkout asks payment engine to authorize a transaction. Aliases:payment auth requested
payment id (DATA-PAYMENT-ID)	Stable identifier used to correlate a payment across services and records.
payment is declined (EVT-PAYMENT-DECLINED)	Event indicating authorization was rejected and customer decline handling must execute. Aliases:payment declined
payment review is pending (STATE-REVIEW-PENDING)	Transaction state requiring manual support review before final disposition. Aliases:review pending
payments (FG-PAYMENTS)	Functional group for checkout and authorization capabilities. Aliases:payments domain, payments system

Term	Definition
platform (FG-PLATFORM)	Functional group for provisioning and platform operations capabilities. Aliases:platform engineering
platform operator (ACT-PLATFORM-OPERATOR)	Role responsible for GitOps and platform lifecycle operations. Aliases:platform ops
postgresql client driver (REF-POSTGRES-DRIVER)	Third-party library used for application data persistence connectivity. Aliases:postgres driver
referenced element (EM-REFERENCED-ELEMENT)	An architecture-relevant external, platform, or third-party dependency represented by role.
replayed authorization callback (AV-REPLAYED-AUTH-CALLBACK)	Replay of a valid callback to induce duplicate or unauthorized state transitions. Aliases:callback replay
risk score (DATA-RISK-SCORE)	Numeric fraud score produced by risk evaluation logic.
risk score is high (EVT-RISK-HIGH)	Event emitted when computed fraud risk exceeds the configured high-risk threshold. Aliases:high risk
risk scoring (FU-RISK-SCORING)	Evaluates transaction risk and returns score classification. Aliases:fraud scoring
risk threshold is exceeded (COND-RISK-THRESHOLD-EXCEEDED)	Condition indicating risk score crossed a policy threshold for escalation. Aliases:risk above threshold
runtime element (EM-RUNTIME-ELEMENT)	An inferred runtime realization element discovered from infrastructure and deployment sources.
support agent (ACT-SUPPORT)	Operations role that investigates declines and high-risk review cases. Aliases:support
support agent is notified (EVT-SUPPORT-NOTIFIED)	Event indicating manual review notification was sent to support operations. Aliases:support notified
support review (FU-SUPPORT-REVIEW)	Supports manual review and escalation handling for risky transactions. Aliases:manual review
upward linking (EM-UPWARD-LINKING)	Rule where runtime and code elements point to stable functional groups/units; authored architecture does not depend on inferred IDs.

5. Functional View

Show authored architecture scope for this view.

5.1. What This View Answers

- What capability areas and responsibilities are intentionally designed?
- How do actors interact with functional units and key dependencies?

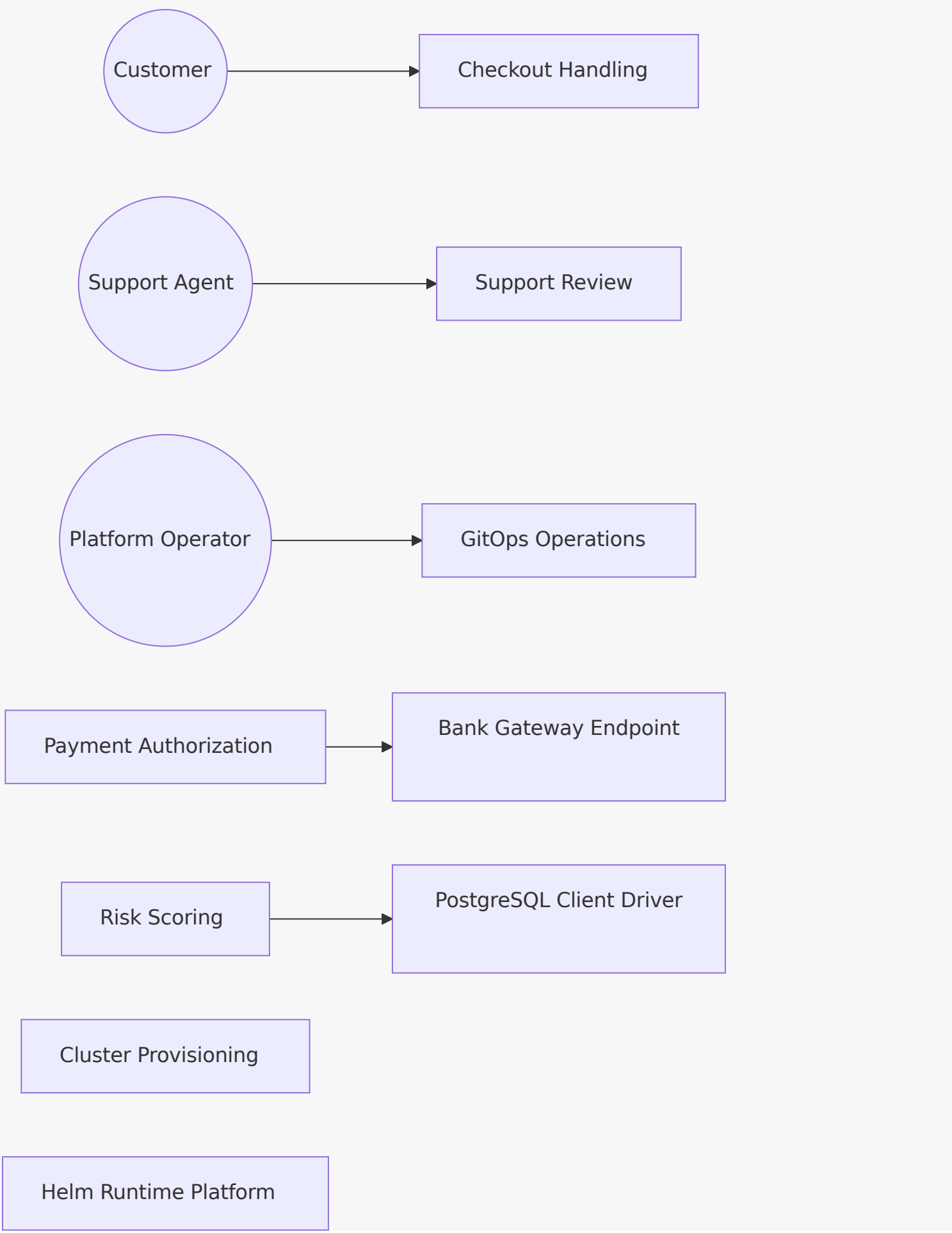
- Which collaborations define the primary system behavior?

5.2. Coverage Gaps

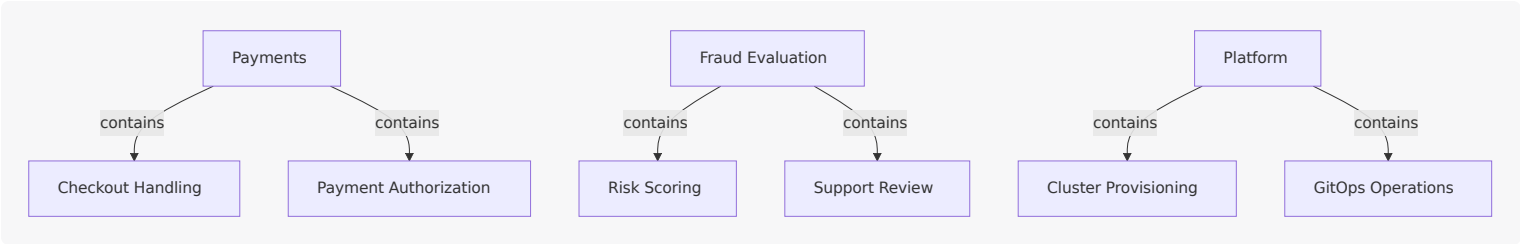
Coverage summary: 6/6 units have direct evidence coverage in this view. - No major coverage gaps detected from current authored and inferred inputs.

5.3. Recommended Next Evidence Additions

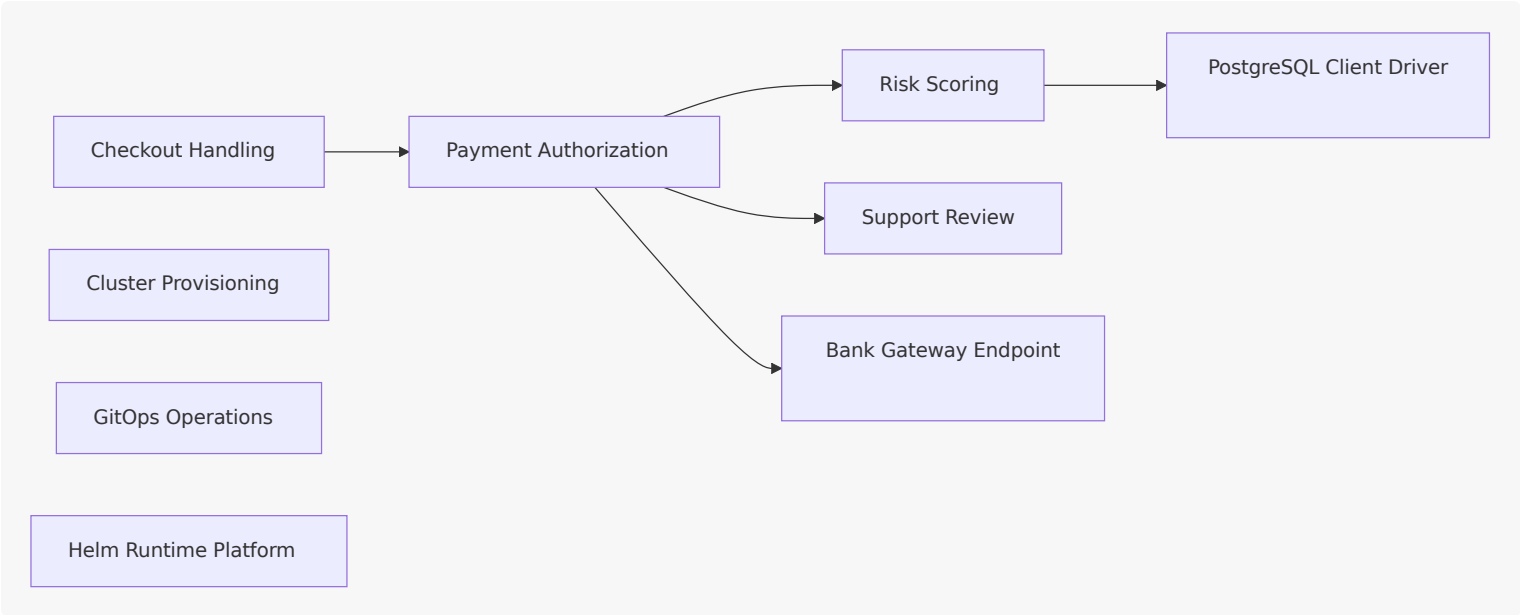
- No immediate evidence additions are required for this view. === Functional Context Diagram


Customer
Checkout Handling
Support Agent
Support Review
Platform Operator
GitOps Operations
Payment Authorization
Bank Gateway Endpoint
Risk Scoring
PostgreSQL Client Driver
Cluster Provisioning
Helm Runtime Platform

5.4. Functional Decomposition Diagram



5.5. Functional Collaboration Diagram



5.6. Functional Groups (View Scoped)

5.6.1. Payments (FG-PAYMENTS)

Description: Core payment checkout and authorization domain.

The payments group defines the product-facing payment journey from user intent to final payment outcome. It owns the consistency of customer messaging and transaction decisions when a payment is attempted, declined, retried, or temporarily unavailable. The group keeps customer flow stable while depending on fraud and external bank capabilities.

Payments Functional Story

payments owns the business flow from customer intent to customer outcome. It coordinates checkout handling and payment authorization so payment authorization is requested consistently while checkout is active. The group is responsible for keeping outcomes deterministic when payment is declined or when bank link is unavailable.

5.6.2. Fraud Evaluation (FG-FRAUD)

Description: Risk scoring and fraud audit domain.

The fraud evaluation group exists to reduce unsafe approvals while preserving throughput for legitimate payments. It combines automated risk scoring with manual support review so uncertain transactions move through a controlled escalation path. The group ensures risk decisions are explainable, auditable, and operationally actionable.

Fraud Evaluation Functional Story

fraud evaluation turns uncertain transaction risk into actionable decisions for payment authorization. It provides risk scoring and support review so high-risk flows are handled as payment review is pending instead of forcing unsafe automatic outcomes. The group must preserve clear transitions from automated scoring to manual handling when support agent is notified.

5.6.3. Platform (FG-PLATFORM)

Description: Cluster provisioning and GitOps operations domain.

The platform group provides the runtime and deployment foundation used by all product capabilities. It owns environment provisioning, namespace and cluster boundaries, and continuous reconciliation behavior. The group enables safe delivery and reliable operations without owning payment business logic.

Platform Functional Story

platform provides the execution foundation and delivery control needed by payments and fraud evaluation. It includes cluster provisioning and gitops operations as explicit architecture capabilities, not background concerns. The group supports normal mode is active operations while enabling controlled lifecycle changes.

5.7. Functional Units (View Scoped)

5.7.1. Risk Scoring (FU-RISK-SCORING)

Group: FG-FRAUD

Risk scoring computes and classifies transaction risk before approval decisions are finalized. It provides a stable scoring contract to authorization and supports audit context for later analysis. The unit is focused on decision quality, consistency, and policy-driven classification behavior.

Risk Scoring Functional Story

risk scoring computes risk score and classification used by payment authorization decisions. It is responsible for consistent threshold behavior when fraud check is enabled. The unit should make risk logic explainable for operators and reviewers.

What it owns: functional responsibility for risk scoring; includes decision and orchestration flow to PostgreSQL Client Driver; primary requirement scope: REQ-PAY-002, REQ-PAY-005 Requirement trigger scope: REQ-PAY-002, REQ-PAY-005 Upstream inputs: Payment Authorization Downstream outcomes: depends_on → PostgreSQL Client Driver Primary dependencies: PostgreSQL Client Driver Primary threat linkage: no explicit attack vector targeting this unit Evidence status: runtime: risk, risk/risk-scorer | code modules: risk_scorer, support

5.7.2. Support Review (FU-SUPPORT-REVIEW)

Group: FG-FRAUD

Support review handles manual decisions for escalated or ambiguous payment cases. It gives support operators context to approve, reject, or request additional verification in a controlled flow. The unit ensures manual intervention remains auditable, policy-constrained, and operationally reliable.

Support Review Functional Story

support review manages human decision flow for escalated transactions. When payment review is pending, it provides context and action paths for support agent to resolve risk outcomes safely. The unit ensures manual decisions remain controlled and consistent.

What it owns: functional responsibility for support review; primary requirement scope: REQ-PAY-004 Requirement trigger scope: REQ-PAY-004 Upstream inputs: Payment Authorization Downstream outcomes: derived unit outcomes from authored mappings Primary dependencies: none Primary threat linkage: no explicit attack vector targeting this unit Evidence status: authored unit with no direct derived runtime/code evidence yet

5.7.3. Checkout Handling (FU-CHECKOUT)

Group: FG-PAYMENTS

Checkout handling is the user-facing entrypoint where payment requests are initiated and normalized. It validates request shape, preserves transaction context, and returns clear customer feedback for each outcome. It delegates decision logic to payment authorization while protecting the user experience boundary.

Checkout Handling Functional Story

checkout handling turns customer input into a valid payment initiation sequence. It keeps customer-facing messaging coherent across success, payment is declined, and bank link is unavailable outcomes. The unit is the main interaction surface for customer during checkout is active.

What it owns: functional responsibility for checkout handling; includes decision and orchestration flow to Payment Authorization; primary requirement scope: REQ-PAY-001, REQ-PAY-003, REQ-PAY-006 Requirement trigger scope: REQ-PAY-001, REQ-PAY-003, REQ-PAY-006 Upstream inputs: none Downstream outcomes: depends_on → Payment Authorization Primary dependencies: Payment Authorization Primary threat linkage: Malicious API Request Evidence status: runtime: payments, payments/checkout-api | code modules: checkout_api

5.7.4. Payment Authorization (FU-PAYMENT-AUTHORIZATION)

Group: FG-PAYMENTS

Payment authorization orchestrates the final transaction decision path. It coordinates fraud scoring, external bank interactions, and escalation to support review when needed. The unit returns deterministic outcomes so downstream behavior remains consistent and testable.

Payment Authorization Functional Story

payment authorization is the policy orchestration core for transaction decisions. It receives payment authorization is requested, calls risk scoring, and evaluates external bank outcomes before returning a single decision path. The unit owns how review escalation and fallback are selected.

What it owns: functional responsibility for payment authorization; includes decision and orchestration flow to Bank Gateway Endpoint and decision and orchestration flow to Risk Scoring; primary requirement scope: REQ-PAY-001, REQ-PAY-004, REQ-PAY-006 Requirement trigger scope: REQ-PAY-001, REQ-PAY-004, REQ-PAY-006 Upstream inputs: Checkout Handling Downstream outcomes: depends_on → Bank Gateway Endpoint; depends_on → Risk Scoring; depends_on → Support Review Primary dependencies: Bank Gateway Endpoint, Risk Scoring, Support Review Primary threat linkage: Replayed Authorization Callback Evidence status: runtime: payments/payment-engine | code modules: domain, payment_engine

5.7.5. Cluster Provisioning (FU-CLUSTER-PROVISIONING)

Group: FG-PLATFORM

Cluster provisioning creates the runtime substrate used by all application workloads. It establishes cluster and namespace structure, baseline controls, and environment-level readiness. This unit is responsible for predictable, repeatable infrastructure foundations.

Cluster Provisioning Functional Story

cluster provisioning creates the baseline environment where application workloads can run safely. It defines cluster and namespace structure needed for payments and fraud evaluation operation. The unit owns repeatable environment creation as a core capability.

What it owns: functional responsibility for cluster provisioning Requirement trigger scope: no explicit authored requirement trigger Upstream inputs: none Downstream outcomes: derived unit outcomes from authored mappings Primary dependencies: none Primary threat linkage: no explicit attack vector targeting this unit Evidence status: runtime: payments

5.7.6. GitOps Operations (FU-GITOPS-OPERATIONS)

Group: FG-PLATFORM

GitOps operations continuously reconciles intended release state with actual runtime state. It governs rollout flow, drift correction, and operational delivery confidence across payment and fraud workloads. The unit provides safe and observable release behavior as an ongoing operational capability.

Gitops Operations Functional Story

gitops operations governs how declared release intent becomes running state. It owns reconciliation behavior that keeps system drift under control across environments. The unit is responsible for predictable delivery behavior over time.

What it owns: functional responsibility for gitops operations Requirement trigger scope: no explicit authored requirement trigger Upstream inputs: none Downstream outcomes: derived unit outcomes from authored mappings Primary dependencies: none Primary threat linkage: no explicit attack vector targeting this unit Evidence status: runtime: flux-system/payments-apps, flux-system/payments-sample-source, flux_system

6. Runtime View

Show inferred runtime interaction and containment mapped to authored unit boundaries. This is a derived runtime view and may be partial when runtime artifacts are missing.

6.1. What This View Answers

- Which runtime elements interact on the request path?
- What API edges and ports are currently evidenced?
- Where is direct runtime evidence still missing?

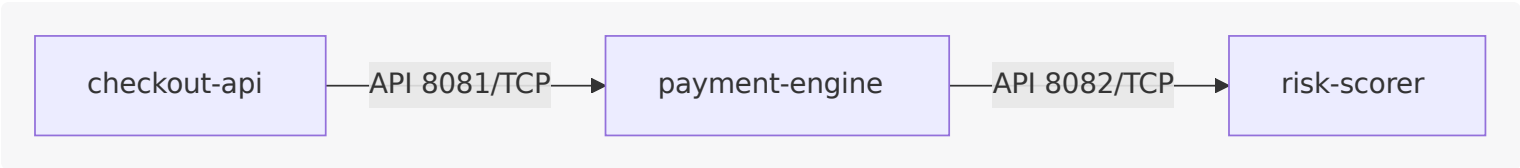
6.2. Coverage Gaps

Coverage summary: 3/4 units have direct evidence coverage in this view. - Support Review has no direct inferred runtime/deployment evidence yet.

6.3. Recommended Next Evidence Additions

- Add explicit owner annotations on runtime/deployment artifacts (for example `engineering-model.dev/owner-unit: FU-*`).
- Expose service protocol/port metadata in Helm or manifest values to enrich API-edge inference.
- Capture missing runtime units by adding discoverable workload artifacts under the scanned IaC/runtime roots. === Runtime API Dependency Graph

> **NOTE**
> This graph is inferred from available deployment/runtime artifacts and may be partial. Missing edges or protocol details indicate missing derived evidence, not necessarily missing runtime behavior.



6.4. Functional Groups (View Scoped)

6.4.1. Payments (FG-PAYMENTS)

Payments Runtime Story

payments runs the customer-facing ingress and transaction orchestration path. Runtime behavior keeps payment id context through service calls so operators can follow one transaction across components. The group must support low-latency customer responses while still waiting for risk and external dependency signals.

6.4.2. Fraud Evaluation (FG-FRAUD)

Fraud Evaluation Runtime Story

fraud evaluation runs scoring and escalation paths that sit directly on the authorization critical path. Runtime responses must be consistent and fast enough to avoid customer-facing instability while still preserving evidence for audit. The group emits clear high-risk signals when risk score is high.

6.5. Functional Units (View Scoped)

6.5.1. Risk Scoring (FU-RISK-SCORING)

Group: FG-FRAUD

Risk Scoring Runtime Story

risk scoring runs as a deterministic dependency service on the transaction path. It should return stable response shapes and clear error semantics so authorization can handle failures safely. Runtime performance must support customer-facing latency expectations.

Runtime Aspect	View-Scoped Detail
Runtime interactions	Consumes: Payment Authorization; Produces: depends_on → PostgreSQL Client Driver
Runtime dependencies	PostgreSQL Client Driver
Inferred runtime evidence	runtime: risk, risk/risk-scorer

6.5.2. Support Review (FU-SUPPORT-REVIEW)

Group: FG-FRAUD

Support Review Runtime Story

support review runtime favors operator clarity and auditability over raw throughput. It should maintain strong linkage between escalation events and operator actions. Runtime transitions must be explicit and reversible where policy allows.

Runtime Aspect	View-Scoped Detail
Runtime interactions	Consumes: Payment Authorization; Produces: derived unit outcomes from authored mappings
Runtime dependencies	none
Inferred runtime evidence	authored unit with no direct derived runtime/code evidence yet

6.5.3. Checkout Handling (FU-CHECKOUT)

Group: FG-PAYMENTS

Checkout Handling Runtime Story

checkout handling serves as the ingress runtime path and forwards normalized requests to payment authorization. It must preserve payment id context and produce stable request contracts under load. Runtime behavior is intentionally thin in decision policy to keep boundaries clean.

Runtime Aspect	View-Scoped Detail
Runtime interactions	Consumes: none; Produces: depends_on → Payment Authorization
Runtime dependencies	Payment Authorization

Runtime Aspect	View-Scoped Detail
Inferred runtime evidence	runtime: payments, payments/checkout-api

6.5.4. Payment Authorization (FU-PAYMENT-AUTHORIZATION)

Group: FG-PAYMENTS

Payment Authorization Runtime Story

payment authorization coordinates runtime dependencies to risk scoring and bank gateway endpoint. It must maintain deterministic ordering and idempotent behavior during retries and transient faults. Runtime observability should show which branch produced the final outcome.

Runtime Aspect	View-Scoped Detail
Runtime interactions	Consumes: Checkout Handling; Produces: depends_on → Bank Gateway Endpoint; depends_on → Risk Scoring; depends_on → Support Review
Runtime dependencies	Bank Gateway Endpoint, Risk Scoring, Support Review
Inferred runtime evidence	runtime: payments/payment-engine

7. Deployment View

Show inferred deployment artifacts and ownership mapping to authored units. This view emphasizes deployment relationships and platform operations.

7.1. What This View Answers

- How are deployment artifacts linked from source to runtime targets?
- What platform operations connect actors to deployment control points?
- Which units still lack deployment/runtime artifact evidence?

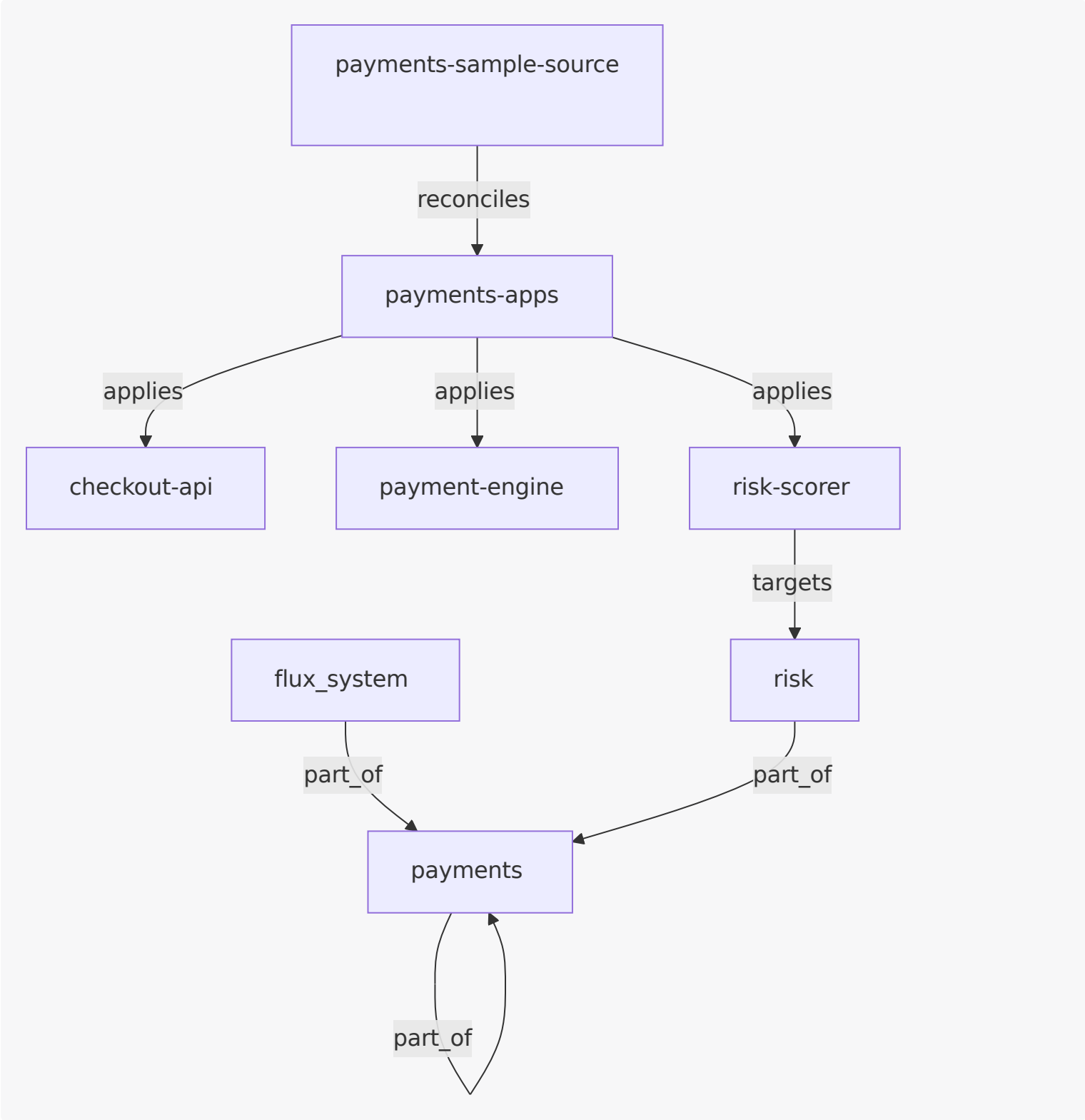
7.2. Coverage Gaps

Coverage summary: 5/6 units have direct evidence coverage in this view. - Support Review has no direct inferred runtime/deployment evidence yet.

7.3. Recommended Next Evidence Additions

- Add owner metadata to Flux/Kustomization/Helm objects for clearer platform ownership mapping.
- Ensure deployment control artifacts (source, kustomization, release) are all included in scanned directories.
- Add deterministic naming conventions for release and namespace objects to improve cross-artifact linking. === Deployment Artifact Relationship Graph

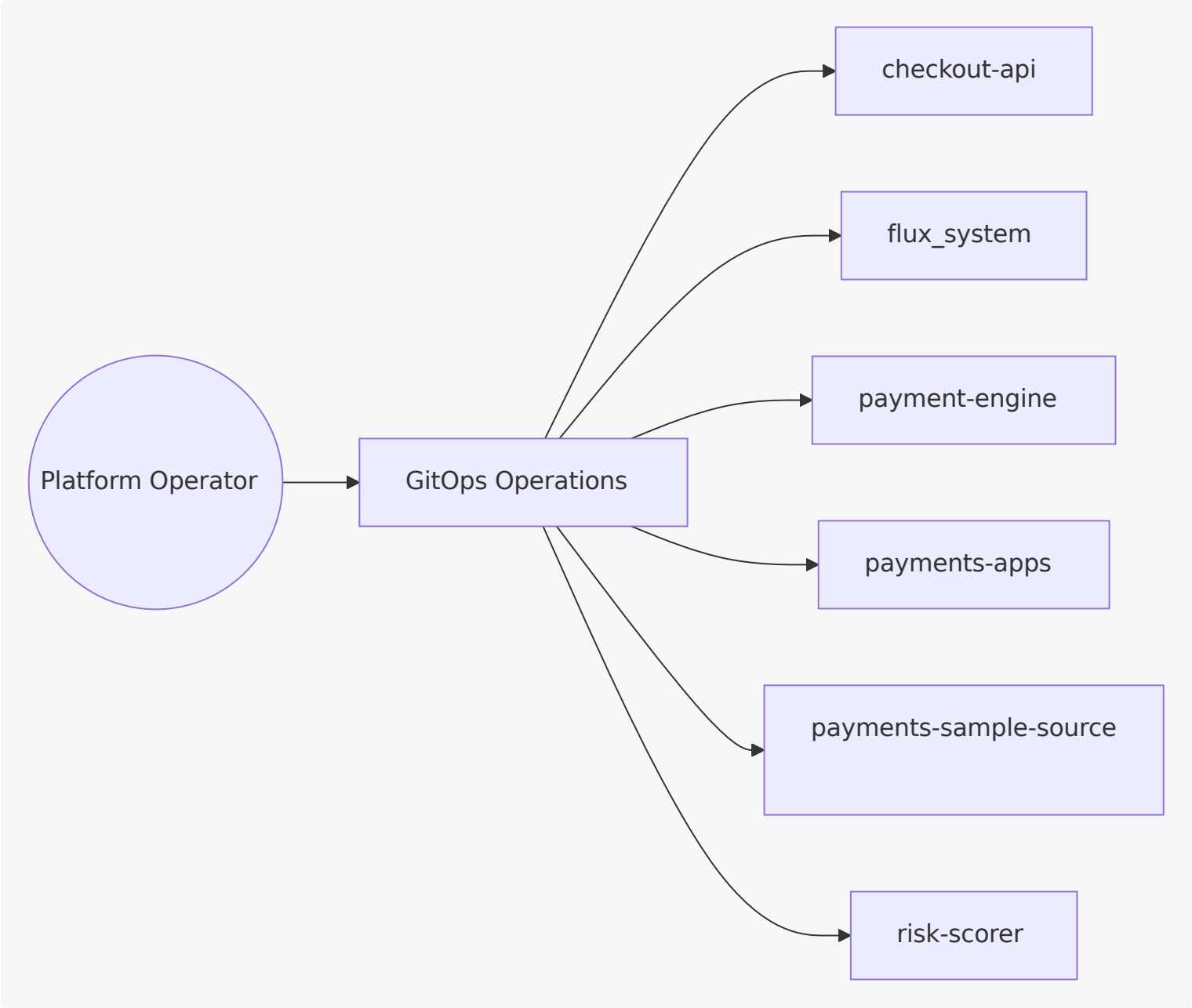
> **NOTE**
> This graph is inferred from available deployment artifacts (for example Terraform, Flux, Helm) and may be partial when source metadata is incomplete.



7.4. Platform Operations Graph

NOTE

Platform operations links are inferred from authored actor mappings plus discovered deployment control-plane artifacts.



7.5. Functional Groups (View Scoped)

7.5.1. Payments (FG-PAYMENTS)

Payments Deployment Story

payments is deployed as independent release paths for checkout handling and payment authorization to reduce rollout blast radius. This allows customer-interface changes and authorization-policy changes to move at different speeds without breaking contracts. Deployment readiness depends on bank link is available and healthy fraud dependencies.

7.5.2. Fraud Evaluation (FG-FRAUD)

Fraud Evaluation Deployment Story

fraud evaluation is deployed as dedicated runtime units so scoring policy and review workflow can be released with targeted controls. Separate deployment paths let teams stage changes to scoring behavior before broad rollout. Operational deployment decisions are guided by risk sensitivity rather than pure throughput.

7.5.3. Platform (FG-PLATFORM)

Platform Deployment Story

platform deployment establishes and governs cluster, namespace, and release-controller behavior before application rollouts proceed. Terraform and GitOps artifacts form the operational path that turns desired state into running systems. This deployment layer is the control surface for safe system evolution.

7.6. Functional Units (View Scoped)

7.6.1. Risk Scoring (FU-RISK-SCORING)

Group: FG-FRAUD

Risk Scoring Deployment Story

risk scoring is deployed with controlled rollout strategy because policy changes can impact approval rates immediately. Teams should stage and observe behavior before broad promotion. Deployment observability should capture score distribution and high-risk frequency.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	REQ-PAY-002, REQ-PAY-005
Deployment dependencies	PostgreSQL Client Driver
Inferred deployment evidence	runtime: risk, risk/risk-scorer

7.6.2. Support Review (FU-SUPPORT-REVIEW)

Group: FG-FRAUD

Support Review Deployment Story

support review deployment should isolate privileged workflows from public-facing ingress components. Release strategy must preserve in-flight review sessions during upgrades. Operational monitoring should focus on queue health and review completion latency.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	REQ-PAY-004
Deployment dependencies	none
Inferred deployment evidence	authored unit with no direct derived runtime/code evidence yet

7.6.3. Checkout Handling (FU-CHECKOUT)

Group: FG-PAYMENTS

Checkout Handling Deployment Story

checkout handling is deployed as an independently releasable workload so UX changes can ship safely. Rollouts should prioritize quick rollback and compatibility with existing authorization contracts. Deployment health is measured by request stability and clear fallback messaging.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	REQ-PAY-001, REQ-PAY-003, REQ-PAY-006
Deployment dependencies	Payment Authorization
Inferred deployment evidence	runtime: payments, payments/checkout-api

7.6.4. Payment Authorization (FU-PAYMENT-AUTHORIZATION)

Group: FG-PAYMENTS

Payment Authorization Deployment Story

payment authorization is deployed separately from checkout handling so policy changes can be staged carefully. Deployment readiness includes dependency health checks and rollback confidence. Releases should protect transaction continuity during rollout windows.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	REQ-PAY-001, REQ-PAY-004, REQ-PAY-006
Deployment dependencies	Bank Gateway Endpoint, Risk Scoring, Support Review
Inferred deployment evidence	runtime: payments/payment-engine

7.6.5. Cluster Provisioning (FU-CLUSTER-PROVISIONING)

Group: FG-PLATFORM

Cluster Provisioning Deployment Story

cluster provisioning is deployed through controlled infrastructure changes and should include drift awareness and rollback planning. Deployment operations must be deterministic because mistakes propagate broadly. Safe change sequencing is a core requirement.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	no explicit authored requirement trigger
Deployment dependencies	none
Inferred deployment evidence	runtime: payments

7.6.6. GitOps Operations (FU-GITOPS-OPERATIONS)

Group: FG-PLATFORM

Gitops Operations Deployment Story

gitops operations deployment defines the control plane used to release and reconcile application artifacts. It must support safe promotion, controlled rollback, and policy gates. Operational resilience depends on this path being reliable and observable.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	no explicit authored requirement trigger
Deployment dependencies	none
Inferred deployment evidence	runtime: flux-system/payments-apps, flux-system/payments-sample-source, flux_system

8. Realization View

Show inferred implementation structure and dependencies mapped to authored architecture. This is a realization-oriented code evidence view.

8.1. What This View Answers

- Which code ownership boundaries map to functional units?
- What first-party and external libraries are evidenced?
- Which units still lack direct code evidence?

8.2. Coverage Gaps

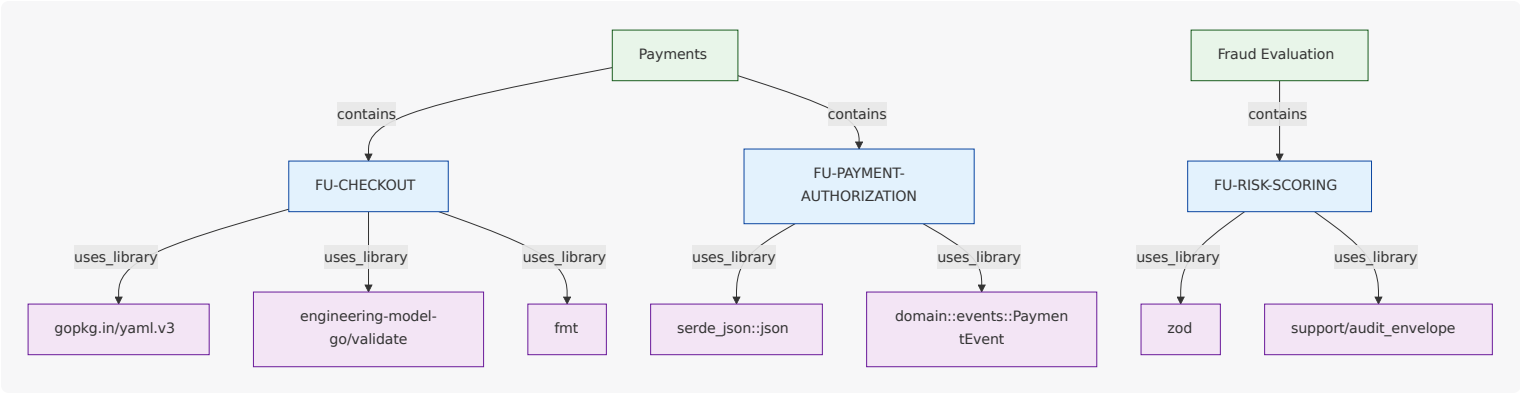
Coverage summary: 3/4 units have direct evidence coverage in this view. - Support Review has no direct inferred code ownership evidence yet.

8.3. Recommended Next Evidence Additions

- Add coarse package/module ownership annotations or ownership mappings for units lacking code evidence.
- Ensure source roots include all first-party modules that realize authored units.
- Add dependency metadata (manifests/imports) where missing to improve library usage evidence.

8.4. Code Dependency Graph

> **NOTE** This view focuses on inferred ownership and library-usage realization evidence. It is not a full source-level call graph.



8.5. Functional Groups (View Scoped)

8.5.1. Payments (FG-PAYMENTS)

Payments Code Ownership Story

payments ownership is coarse and stable at module boundaries, not function boundaries. Teams own checkout handling and payment authorization code areas as long-lived architecture responsibilities, while individual symbols only support requirement evidence. This keeps architecture ownership stable even when internal implementation changes.

8.5.2. Fraud Evaluation (FG-FRAUD)

Fraud Evaluation Code Ownership Story

ownership in fraud evaluation is organized by durable module boundaries for scoring and review responsibilities. Requirement-level traces remain fine-grained, but ownership stays at package and module scope so architecture does not churn. This keeps accountability clear when scoring logic evolves.

8.6. Functional Units (View Scoped)

8.6.1. Risk Scoring (FU-RISK-SCORING)

Group: FG-FRAUD

Risk Scoring Code Ownership Story

risk scoring ownership stays at module boundary to keep policy accountability intact. Fine-grained symbols map to requirements but do not define ownership. This allows model and rules refactoring without architecture instability.

Realization ownership boundary: functional responsibility for risk scoring; includes decision and orchestration flow to PostgreSQL Client Driver; primary requirement scope: REQ-PAY-002, REQ-PAY-005 Implementation trigger context: REQ-PAY-002, REQ-PAY-005 Implementation inputs/outputs: Consumes Payment Authorization; Produces depends_on → PostgreSQL Client Driver Implementation dependencies: PostgreSQL Client Driver Realization evidence: runtime: risk, risk/risk-scorer | code modules: risk_scorer, support

8.6.2. Support Review (FU-SUPPORT-REVIEW)

Group: FG-FRAUD

Support Review Code Ownership Story

support review ownership is kept at module boundary for review workflow and notification logic. Shared utilities may exist, but ownership of decision workflow remains explicit. This prevents ambiguous accountability for manual outcomes.

Realization ownership boundary: functional responsibility for support review; primary requirement scope: REQ-PAY-004 Implementation trigger context: REQ-PAY-004 Implementation inputs/outputs: Consumes Payment Authorization; Produces derived unit outcomes from authored mappings Implementation dependencies: none Realization evidence: authored unit with no direct derived runtime/code evidence yet

8.6.3. Checkout Handling (FU-CHECKOUT)

Group: FG-PAYMENTS

Checkout Handling Code Ownership Story

checkout handling ownership is anchored at module and package level in the checkout code boundary. Teams can refactor methods without changing architecture ownership because ownership is not symbol-based. Requirement traces remain at symbol level only for evidence.

Realization ownership boundary: functional responsibility for checkout handling; includes decision and orchestration flow to Payment Authorization; primary requirement scope: REQ-PAY-001, REQ-PAY-003, REQ-PAY-006 Implementation trigger context: REQ-PAY-001, REQ-PAY-003, REQ-PAY-006 Implementation inputs/outputs: Consumes none; Produces depends_on → Payment Authorization Implementation dependencies: Payment Authorization Realization evidence: runtime: payments, payments/checkout-api | code modules: checkout_api

8.6.4. Payment Authorization (FU-PAYMENT-AUTHORIZATION)

Group: FG-PAYMENTS

Payment Authorization Code Ownership Story

payment authorization ownership is maintained at the payment engine module boundary. Internal algorithms and helper functions can change frequently without changing ownership contracts. This preserves team accountability for orchestration behavior.

Realization ownership boundary: functional responsibility for payment authorization; includes decision and orchestration flow to Bank Gateway Endpoint and decision and orchestration flow to Risk Scoring; primary requirement scope: REQ-PAY-001, REQ-PAY-004, REQ-PAY-006 Implementation trigger context: REQ-PAY-001, REQ-PAY-004, REQ-PAY-006 Implementation inputs/outputs: Consumes Checkout Handling; Produces depends_on → Bank Gateway Endpoint; depends_on → Risk Scoring; depends_on → Support Review Implementation dependencies: Bank Gateway Endpoint, Risk Scoring, Support Review Realization evidence: runtime: payments/payment-engine | code modules: domain, payment_engine

9. Security View

Show inferred exposure and dependency risk points aligned to unit boundaries, focused on attack paths and security evidence.

9.1. What This View Answers

- Which attack vectors currently target authored units and dependencies?
- What security-relevant observability evidence is present?
- Where is explicit threat mapping incomplete?

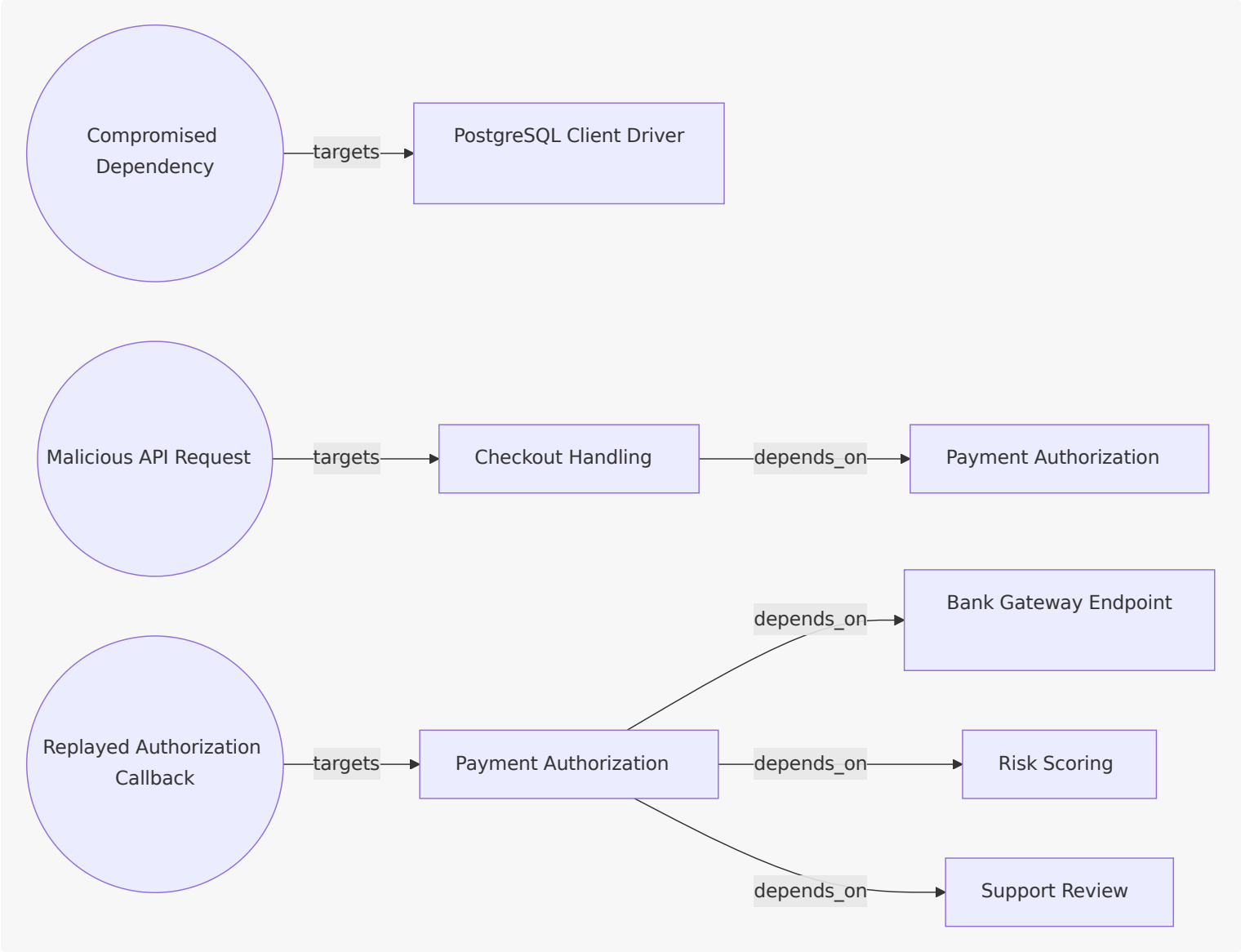
9.2. Coverage Gaps

Coverage summary: 2/6 units have direct evidence coverage in this view. - Cluster Provisioning has no explicit authored attack-vector mapping yet. - GitOps Operations has no explicit authored attack-vector mapping yet. - Risk Scoring has no explicit authored attack-vector mapping yet. - Support Review has no explicit authored attack-vector mapping yet.

9.3. Recommended Next Evidence Additions

- Add explicit authored attack-vector mappings for units currently marked without threat linkage.
- Add security signal ownership metadata (logs/alerts/events) for unresolved observability evidence.
- Map additional security-relevant dependencies and exposures into authored mappings for traceable coverage. === Attack Vector Path Graph

NOTE	Security relationships shown here are explicitly modeled or inferred from available evidence and may be incomplete. Absence of a path does not imply absence of risk.



9.4. Security Logging and Observability

Signal	Layer	Owner	Evidence
application security telemetry hooks	code	FU-RISK-SCORING	code support/audit_envelope.ts
fraud and abuse detection code paths	code	FU-RISK-SCORING	code risk_scorer.ts
deployment and platform audit events	deployment	FU-CHECKOUT	helmrelease checkout-api
deployment and platform audit events	deployment	FU-CHECKOUT	namespace payments
deployment and platform audit events	deployment	FU-CLUSTER-PROVISIONING	cluster payments

Signal	Layer	Owner	Evidence
deployment and platform audit events	deployment	FU-GITOPS-OPERATIONS	gitrepository payments-sample-source
deployment and platform audit events	deployment	FU-GITOPS-OPERATIONS	kustomization payments-apps
deployment and platform audit events	deployment	FU-GITOPS-OPERATIONS	namespace flux_system
deployment and platform audit events	deployment	FU-PAYMENT-AUTHORIZATION	helmrelease payment-engine
deployment and platform audit events	deployment	FU-RISK-SCORING	helmrelease risk-scorer
deployment and platform audit events	deployment	FU-RISK-SCORING	namespace risk
abuse and fraud decision logs	runtime	FU-RISK-SCORING	helmrelease risk-scorer
abuse and fraud decision logs	runtime	FU-RISK-SCORING	namespace risk

9.5. Functional Groups (View Scoped)

9.5.1. Payments (FG-PAYMENTS)

Payments Security Story

payments protects the most exposed trust boundaries in the system. The group must defend against malicious api request and replayed authorization callback while preserving safe behavior when external responses are degraded. Security controls prioritize validation, idempotency, and predictable failure handling.

9.5.2. Fraud Evaluation (FG-FRAUD)

Fraud Evaluation Security Story

fraud evaluation protects decision integrity and review integrity. It must resist compromised dependency influence, malformed risk inputs, and unauthorized manual overrides. Security policy requires traceable review actions and trustworthy score lineage.

9.5.3. Platform (FG-PLATFORM)

Platform Security Story

platform is the primary baseline security layer for the entire system. It must enforce hardened defaults, namespace isolation, and trusted reconciliation so downstream units inherit safer operating conditions. Security failures here are systemic, so controls must be preventative and continuously verified.

9.6. Functional Units (View Scoped)

9.6.1. Risk Scoring (FU-RISK-SCORING)

Group: FG-FRAUD

Risk Scoring Security Story

risk scoring must detect and resist manipulated or poisoned inputs that could lower risk decisions incorrectly. It should enforce strict validation and safe defaults on uncertain data. Security evidence must support post-incident reconstruction of score decisions.

Threat model focus: no explicit attack vector targeting this unit Security trigger context: REQ-PAY-002, REQ-PAY-005 Security-relevant dependencies: PostgreSQL Client Driver Security evidence: runtime: risk, risk/risk-scorer | code modules: risk_scorer, support

9.6.2. Support Review (FU-SUPPORT-REVIEW)

Group: FG-FRAUD

Support Review Security Story

support review security requires strong operator authentication, least privilege, and non-repudiable action logging. Unauthorized override and hidden edits must be prevented by design. Every decision should remain auditable end to end.

Threat model focus: no explicit attack vector targeting this unit Security trigger context: REQ-PAY-004 Security-relevant dependencies: none Security evidence: authored unit with no direct derived runtime/code evidence yet

9.6.3. Checkout Handling (FU-CHECKOUT)

Group: FG-PAYMENTS

Checkout Handling Security Story

checkout handling must aggressively validate input and defend public ingress from malicious api request patterns. It should throttle abusive behavior without degrading legitimate customer flows. Security design emphasizes safe rejection and traceable decisions.

Threat model focus: Malicious API Request Security trigger context: REQ-PAY-001, REQ-PAY-003, REQ-PAY-006 Security-relevant dependencies: Payment Authorization Security evidence: runtime: payments, payments/checkout-api | code modules: checkout_api

9.6.4. Payment Authorization (FU-PAYMENT-AUTHORIZATION)

Group: FG-PAYMENTS

Payment Authorization Security Story

payment authorization must prevent replayed authorization callback abuse and protect integrity of decision transitions. It should normalize external errors so attackers cannot infer sensitive policy details. Security controls also require auditable decision provenance.

Threat model focus: Replayed Authorization Callback Security trigger context: REQ-PAY-001, REQ-PAY-004, REQ-PAY-006 Security-relevant dependencies: Bank Gateway Endpoint, Risk Scoring, Support Review Security evidence: runtime: payments/payment-engine | code modules: domain, payment_engine

9.6.5. Cluster Provisioning (FU-CLUSTER-PROVISIONING)

Group: FG-PLATFORM

Cluster Provisioning Security Story

cluster provisioning security establishes hardened defaults and isolation guarantees inherited by all other units. It should enforce secure baseline policies before workloads are admitted. Security drift at this layer must be detected quickly.

Threat model focus: no explicit attack vector targeting this unit Security trigger context: no explicit authored requirement trigger Security-relevant dependencies: none Security evidence: runtime: payments

9.6.6. GitOps Operations (FU-GITOPS-OPERATIONS)

Group: FG-PLATFORM

Gitops Operations Security Story

gitops operations security protects source trust, reconciliation authority, and rollout safety. It must prevent unauthorized changes from reaching runtime state and enforce policy before apply. This is a core defense against compromised dependency and supply-chain risk.

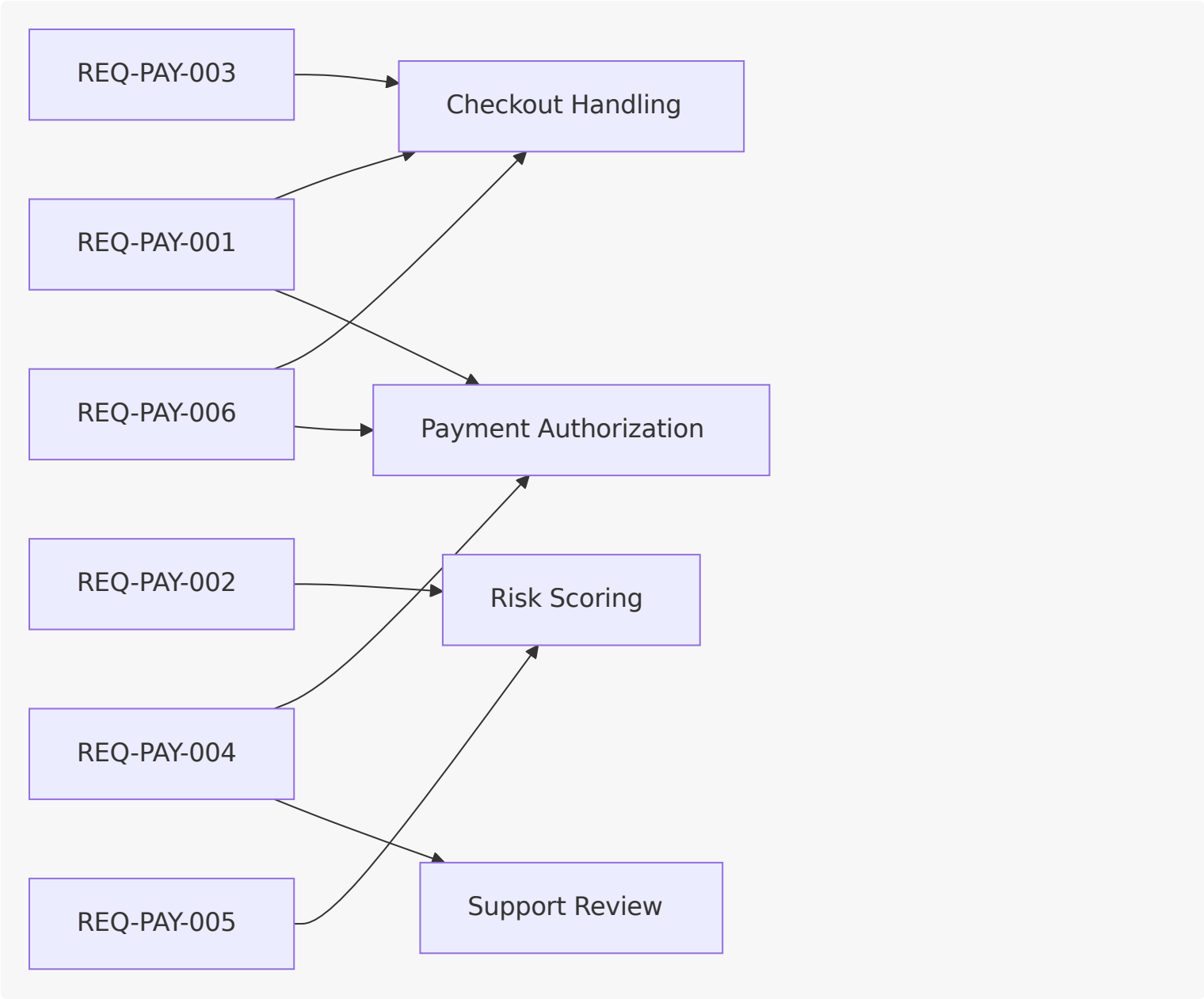
Threat model focus: no explicit attack vector targeting this unit Security trigger context: no explicit authored requirement trigger Security-relevant dependencies: none Security evidence: runtime: flux-system/payments-apps, flux-system/payments-sample-source, flux_system

10. Generated Evidence Appendix

This appendix contains generated traceability material and inferred-reference inventories.

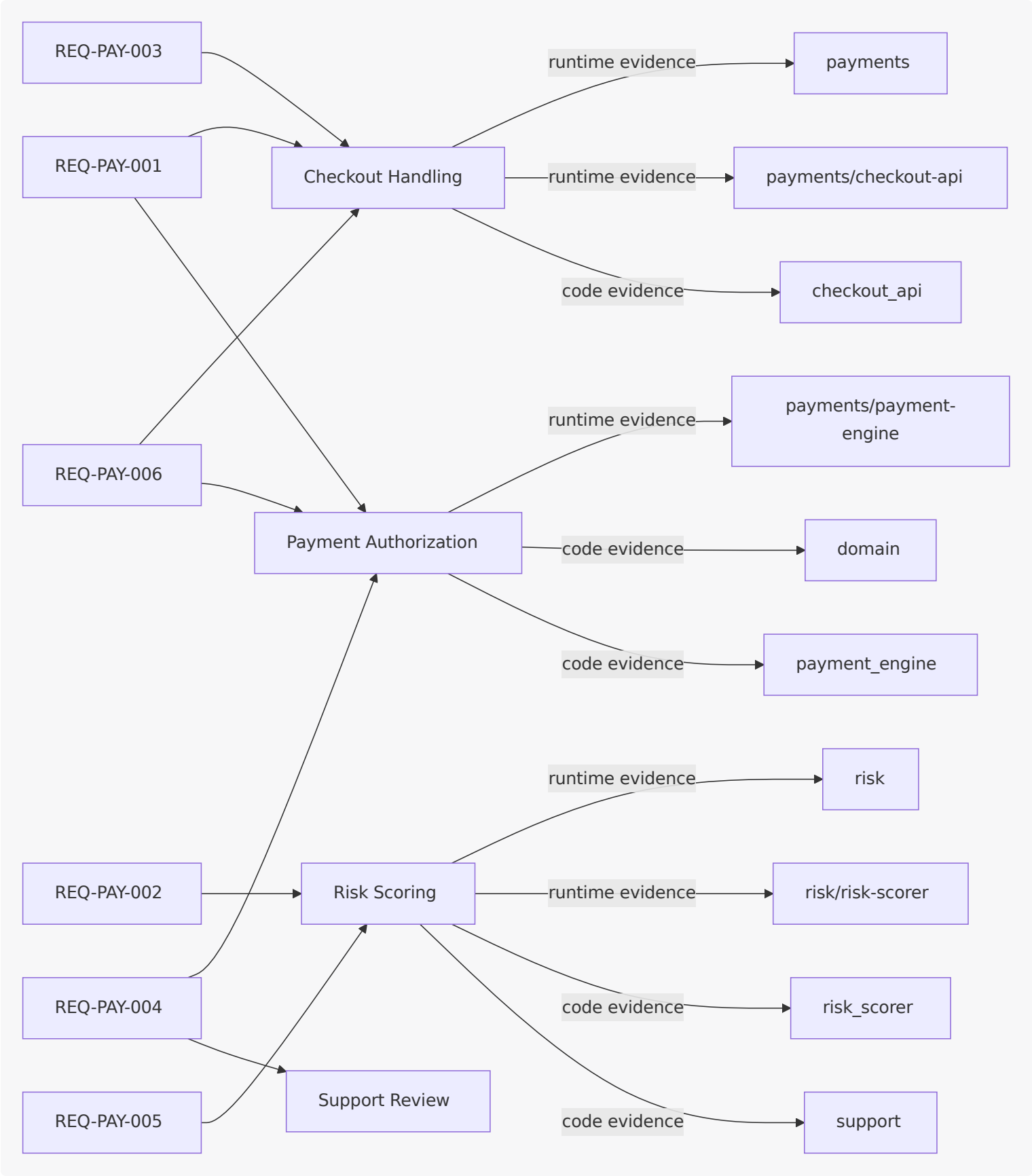
11. Requirement Alignment

Requirement-to-unit overview:



Show requirement-to-unit mappings inferred from appliesTo and authored architecture ownership boundaries.

11.1. Cross-Layer Coverage



11.2. Requirement Details

11.2.1. REQ-PAY-001

While checkout is active, when payment authorization is requested, the payments system shall create a payment session and request authorization from bank gateway. Notes: Core authorization path from checkout to bank gateway.

11.2.2. REQ-PAY-002

Where fraud check is enabled, when payment authorization is requested, the fraud detection system shall calculate a risk score before approval. Notes: Fraud scoring gate before approval.

11.2.3. REQ-PAY-003

If payment is declined, then the payments system shall present a decline reason and a retry option to customer. Notes: Decline handling and customer feedback.

11.2.4. REQ-PAY-004

While checkout is active, when payment authorization is requested and risk score is high, the payments system shall place the payment in review pending state and notify support agent. Notes: Escalation flow for high-risk payments.

11.2.5. REQ-PAY-005

Where fraud check is enabled, when payment authorization is requested, the payments system shall persist an audit record with payment id and risk score. Notes: Auditability for authorization decisions.

11.2.6. REQ-PAY-006

If bank link is unavailable, then the payments system shall present a temporary unavailable message to customer. Notes: Fallback behavior when external banking path is down.

12. Reference Index

12.1. Authored References

ACT-CUSTOMER

Kind: Actor Name: Customer

Starts checkout and confirms payment.

ACT-PLATFORM-OPERATOR

Kind: Actor Name: Platform Operator

Operates GitOps and platform lifecycle.

ACT-SUPPORT

Kind: Actor Name: Support Agent

Reviews high-risk and declined-payment cases.

AV-COMPROMISED-DEPENDENCY

Kind: Attack Vector Name: Compromised Dependency

Supply chain compromise in external SDK or image.

AV-MALICIOUS-API-REQUEST

Kind: Attack Vector Name: Malicious API Request

Crafted payloads attempting to bypass authorization controls.

AV-REPLAYED-AUTH-CALLBACK

Kind: Attack Vector Name: Replayed Authorization Callback

Replay attack against external authorization callback flow.

FG-FRAUD

Kind: Functional Group Name: Fraud Evaluation

Risk scoring and fraud audit domain.

FG-PAYMENTS

Kind: Functional Group Name: Payments

Core payment checkout and authorization domain.

FG-PLATFORM

Kind: Functional Group Name: Platform

Cluster provisioning and GitOps operations domain.

FU-CHECKOUT

Kind: Functional Unit Name: Checkout Handling

Checkout handling is the user-facing entrypoint where payment requests are initiated and normalized. It validates request shape, preserves transaction context, and returns clear customer feedback for each outcome. It delegates decision logic to payment authorization while protecting the user experience boundary.

FU-CLUSTER-PROVISIONING

Kind: Functional Unit Name: Cluster Provisioning

Cluster provisioning creates the runtime substrate used by all application workloads. It establishes cluster and namespace structure, baseline controls, and environment-level readiness. This unit is responsible for predictable, repeatable infrastructure foundations.

FU-GITOPS-OPERATIONS

Kind: Functional Unit Name: GitOps Operations

GitOps operations continuously reconciles intended release state with actual runtime state. It governs rollout flow, drift correction, and operational delivery confidence across payment and fraud workloads. The unit provides safe and observable release behavior as an ongoing operational capability.

FU-PAYMENT-AUTHORIZATION

Kind: Functional Unit Name: Payment Authorization

Payment authorization orchestrates the final transaction decision path. It coordinates fraud scoring, external bank interactions, and escalation to support review when needed. The unit returns deterministic outcomes so downstream behavior remains consistent and testable.

FU-RISK-SCORING

Kind: Functional Unit Name: Risk Scoring

Risk scoring computes and classifies transaction risk before approval decisions are finalized. It provides a stable scoring contract to authorization and supports audit context for later analysis. The unit is focused on decision quality, consistency, and policy-driven classification behavior.

FU-SUPPORT-REVIEW

Kind: Functional Unit Name: Support Review

Support review handles manual decisions for escalated or ambiguous payment cases. It gives support operators context to approve, reject, or request additional verification in a controlled flow. The unit ensures manual intervention remains auditable, policy-constrained, and operationally reliable.

REF-BANK-GATEWAY-ENDPOINT

Kind: Referenced Element Name: Bank Gateway Endpoint

external_service_endpoint / runtime

REF-HELM-PLATFORM

Kind: Referenced Element Name: Helm Runtime Platform

platform_service / runtime

REF-POSTGRES-DRIVER

Kind: Referenced Element Name: PostgreSQL Client Driver

third_party_library / code

REQ-PAY-001

Kind: Requirement Name: REQ-PAY-001 Resolves to: canonical While checkout is active, when payment authorization is requested, the payments system shall create a payment session and request authorization from bank gateway.

REQ-PAY-002

Kind: Requirement Name: REQ-PAY-002 Resolves to: canonical Where fraud check is enabled, when payment authorization is requested, the fraud detection system shall calculate a risk score before approval.

REQ-PAY-003

Kind: Requirement Name: REQ-PAY-003 Resolves to: canonical If payment is declined, then the payments system shall present a decline reason and a retry option to customer.

REQ-PAY-004

Kind: Requirement Name: REQ-PAY-004 Resolves to: canonical While checkout is active, when payment authorization is requested and risk score is high, the payments system shall place the payment in review pending state and notify support agent.

REQ-PAY-005

Kind: Requirement Name: REQ-PAY-005 Resolves to: canonical Where fraud check is enabled, when payment authorization is requested, the payments system shall persist an audit record with payment id and risk score.

REQ-PAY-006

Kind: Requirement Name: REQ-PAY-006 Resolves to: canonical If bank link is unavailable, then the payments system shall present a temporary unavailable message to customer.

12.2. Catalog References

ACT-CUSTOMER

Kind: Catalog Actor Name: customer Resolves to: canonical End user submitting payment details during checkout.

ACT-PLATFORM-OPERATOR

Kind: Catalog Actor Name: platform operator Resolves to: canonical Role responsible for GitOps and platform lifecycle operations.

ACT-SUPPORT

Kind: Catalog Actor Name: support agent Resolves to: canonical Operations role that investigates declines and high-risk review cases.

3DS required

Kind: Catalog Alias Name: 3DS verification is required Resolves to: canonical Alias of COND-3DS-REQUIRED

auth

Kind: Catalog Alias Name: payment authorization Resolves to: canonical Alias of FU-PAYMENT-AUTHORIZATION

bank API

Kind: Catalog Alias Name: bank gateway endpoint Resolves to: canonical Alias of REF-BANK-GATEWAY-ENDPOINT

bank link down

Kind: Catalog Alias Name: bank link is unavailable Resolves to: canonical Alias of EVT-BANK-LINK-UNAVAILABLE

bank link up

Kind: Catalog Alias Name: bank link is available Resolves to: canonical Alias of STATE-BANK-LINK-UP

buyer

Kind: Catalog Alias Name: customer Resolves to: canonical Alias of ACT-CUSTOMER

callback replay

Kind: Catalog Alias Name: replayed authorization callback Resolves to: canonical Alias of AV-REPLAYED-AUTH-CALLBACK

checkout

Kind: Catalog Alias Name: checkout handling Resolves to: canonical Alias of FU-CHECKOUT

checkout active

Kind: Catalog Alias Name: checkout is active Resolves to: canonical Alias of STATE-CHECKOUT-ACTIVE

day1 provisioning

Kind: Catalog Alias Name: cluster provisioning Resolves to: canonical Alias of FU-CLUSTER-PROVISIONING

day2 operations

Kind: Catalog Alias Name: gitops operations Resolves to: canonical Alias of FU-GITOPS-OPERATIONS

decline code

Kind: Catalog Alias Name: decline reason Resolves to: canonical Alias of DATA-DECLINE-REASON

dependency compromise

Kind: Catalog Alias Name: compromised dependency Resolves to: canonical Alias of AV-COMPROMISED-DEPENDENCY

fraud check

Kind: Catalog Alias Name: fraud check is enabled Resolves to: canonical Alias of FEAT-FRAUD-CHECK

fraud detection system

Kind: Catalog Alias Name: fraud evaluation Resolves to: canonical Alias of FG-FRAUD

fraud domain

Kind: Catalog Alias Name: fraud evaluation Resolves to: canonical Alias of FG-FRAUD

fraud scoring

Kind: Catalog Alias Name: risk scoring Resolves to: canonical Alias of FU-RISK-SCORING

helm

Kind: Catalog Alias Name: helm platform Resolves to: canonical Alias of REF-HELM-PLATFORM

high risk

Kind: Catalog Alias Name: risk score is high Resolves to: canonical Alias of EVT-RISK-HIGH

malicious request

Kind: Catalog Alias Name: malicious api request Resolves to: canonical Alias of AV-MALICIOUS-API-REQUEST

manual review

Kind: Catalog Alias Name: support review Resolves to: canonical Alias of FU-SUPPORT-REVIEW

normal mode

Kind: Catalog Alias Name: normal mode is active Resolves to: canonical Alias of MODE-NORMAL

payment auth requested

Kind: Catalog Alias Name: payment authorization is requested Resolves to: canonical Alias of EVT-PAYMENT-AUTH-REQUESTED

payment declined

Kind: Catalog Alias Name: payment is declined Resolves to: canonical Alias of EVT-PAYMENT-DECLINED

payments domain

Kind: Catalog Alias Name: payments Resolves to: canonical Alias of FG-PAYMENTS

payments system

Kind: Catalog Alias Name: payments Resolves to: canonical Alias of FG-PAYMENTS

platform engineering

Kind: Catalog Alias Name: platform Resolves to: canonical Alias of FG-PLATFORM

platform ops

Kind: Catalog Alias Name: platform operator Resolves to: canonical Alias of ACT-PLATFORM-OPERATOR

postgres driver

Kind: Catalog Alias Name: postgresql client driver Resolves to: canonical Alias of REF-POSTGRES-DRIVER

review pending

Kind: Catalog Alias Name: payment review is pending Resolves to: canonical Alias of STATE-REVIEW-PENDING

risk above threshold

Kind: Catalog Alias Name: risk threshold is exceeded Resolves to: canonical Alias of COND-RISK-THRESHOLD-EXCEEDED

support

Kind: Catalog Alias Name: support agent Resolves to: canonical Alias of ACT-SUPPORT

support notified

Kind: Catalog Alias Name: support agent is notified Resolves to: canonical Alias of EVT-SUPPORT-NOTIFIED

AV-COMPROMISED-DEPENDENCY

Kind: Catalog Attack Vector Name: compromised dependency Resolves to: canonical Supply-chain compromise in runtime image or code dependency.

AV-MALICIOUS-API-REQUEST

Kind: Catalog Attack Vector Name: malicious api request Resolves to: canonical Crafted payload attempting to bypass validation or authorization logic.

AV-REPLAYED-AUTH-CALLBACK

Kind: Catalog Attack Vector Name: replayed authorization callback Resolves to: canonical Replay of a valid callback to induce duplicate or unauthorized state transitions.

COND-3DS-REQUIRED

Kind: Catalog Condition Name: 3DS verification is required Resolves to: canonical Condition indicating strong customer authentication must be performed.

COND-RISK-THRESHOLD-EXCEEDED

Kind: Catalog Condition Name: risk threshold is exceeded Resolves to: canonical Condition indicating risk score crossed a policy threshold for escalation.

DATA-DECLINE-REASON

Kind: Catalog Data Term Name: decline reason Resolves to: canonical Structured reason code or message explaining why authorization failed.

DATA-PAYMENT-ID

Kind: Catalog Data Term Name: payment id Resolves to: canonical Stable identifier used to correlate a payment across services and records.

DATA-RISK-SCORE

Kind: Catalog Data Term Name: risk score Resolves to: canonical Numeric fraud score produced by risk evaluation logic.

EVT-BANK-LINK-UNAVAILABLE

Kind: Catalog Event Name: bank link is unavailable Resolves to: canonical Event raised when payment engine cannot reach the external bank gateway.

EVT-PAYMENT-AUTH-REQUESTED

Kind: Catalog Event Name: payment authorization is requested Resolves to: canonical Event raised when checkout asks payment engine to authorize a transaction.

EVT-PAYMENT-DECLINED

Kind: Catalog Event Name: payment is declined Resolves to: canonical Event indicating authorization was rejected and customer decline handling must execute.

EVT-RISK-HIGH

Kind: Catalog Event Name: risk score is high Resolves to: canonical Event emitted when computed fraud risk exceeds the configured high-risk threshold.

EVT-SUPPORT-NOTIFIED

Kind: Catalog Event Name: support agent is notified Resolves to: canonical Event indicating manual review notification was sent to support operations.

FEAT-FRAUD-CHECK

Kind: Catalog Feature Name: fraud check is enabled Resolves to: canonical Feature flag controlling whether risk scoring is required before authorization.

FG-FRAUD

Kind: Catalog Functional Group Name: fraud evaluation Resolves to: canonical Functional group for risk scoring and fraud review capabilities.

FG-PAYMENTS

Kind: Catalog Functional Group Name: payments Resolves to: canonical Functional group for checkout and authorization capabilities.

FG-PLATFORM

Kind: Catalog Functional Group Name: platform Resolves to: canonical Functional group for provisioning and platform operations capabilities.

FU-CHECKOUT

Kind: Catalog Functional Unit Name: checkout handling Resolves to: canonical Handles checkout session start and customer-facing payment messaging.

FU-CLUSTER-PROVISIONING

Kind: Catalog Functional Unit Name: cluster provisioning Resolves to: canonical Provides cluster-level platform setup and environment bootstrap.

FU-GITOPS-OPERATIONS

Kind: Catalog Functional Unit Name: gitops operations Resolves to: canonical Operates declarative delivery and release management flows.

FU-PAYMENT-AUTHORIZATION

Kind: Catalog Functional Unit Name: payment authorization Resolves to: canonical Coordinates authorization flow and bank gateway interaction.

FU-RISK-SCORING

Kind: Catalog Functional Unit Name: risk scoring Resolves to: canonical Evaluates transaction risk and returns score classification.

FU-SUPPORT-REVIEW

Kind: Catalog Functional Unit Name: support review Resolves to: canonical Supports manual review and escalation handling for risky transactions.

MODE-NORMAL

Kind: Catalog Mode Name: normal mode is active Resolves to: canonical Standard operating mode with full external dependency usage enabled.

REF-BANK-GATEWAY-ENDPOINT

Kind: Catalog Referenced Element Name: bank gateway endpoint Resolves to: canonical External banking endpoint used for card authorization and decline decisions.

REF-HELM-PLATFORM

Kind: Catalog Referenced Element Name: helm platform Resolves to: canonical Runtime packaging and deployment substrate for workloads.

REF-POSTGRES-DRIVER

Kind: Catalog Referenced Element Name: postgresql client driver Resolves to: canonical Third-party library used for application data persistence connectivity.

STATE-BANK-LINK-UP

Kind: Catalog State Name: bank link is available Resolves to: canonical Connectivity state showing bank gateway communication is currently healthy.

STATE-CHECKOUT-ACTIVE

Kind: Catalog State Name: checkout is active Resolves to: canonical Processing state where the user is actively submitting or confirming payment.

STATE-REVIEW-PENDING

Kind: Catalog State Name: payment review is pending Resolves to: canonical Transaction state requiring manual support review before final disposition.

12.3. Inferred Runtime References

payments

Kind: Runtime cluster Owner: FU-CLUSTER-PROVISIONING Source: examples/payments-engineering-sample/infra/terraform/main.tf

flux-system/payments-sample-source

Kind: Runtime gitrepository Owner: FU-GITOPS-OPERATIONS Source: examples/payments-engineering-sample/infra/flux/sources/payments-source.yaml

payments/checkout-api

Kind: Runtime helmrelease Owner: FU-CHECKOUT Source: examples/payments-engineering-sample/infra/flux/releases/checkout-api.yaml

payments/payment-engine

Kind: Runtime helmrelease Owner: FU-PAYMENT-AUTHORIZATION Source: examples/payments-engineering-sample/infra/flux/releases/payment-engine.yaml

risk/risk-scorer

Kind: Runtime helmrelease Owner: FU-RISK-SCORING Source: examples/payments-engineering-sample/infra/flux/releases/risk-scorer.yaml

flux-system/payments-apps

Kind: Runtime kustomization Owner: FU-GITOPS-OPERATIONS Source: examples/payments-engineering-sample/infra/flux/kustomizations/payments-apps.yaml

flux_system

Kind: Runtime namespace Owner: FU-GITOPS-OPERATIONS Source: examples/payments-engineering-sample/infra/terraform/main.tf

payments

Kind: Runtime namespace Owner: FU-CHECKOUT Source: examples/payments-engineering-sample/infra/terraform/main.tf

risk

Kind: Runtime namespace Owner: FU-RISK-SCORING Source: examples/payments-engineering-sample/infra/terraform/main.tf

12.4. Inferred Code References

gopkg.in/yaml.v3

Kind: Code library_external Owner: FU-CHECKOUT Source: checkout_api.go

serde_json::json

Kind: Code library_external Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs

zod

Kind: Code library_external Owner: FU-RISK-SCORING Source: risk_scorer.ts

./support/audit_envelope

Kind: Code library_first_party Owner: FU-RISK-SCORING Source: risk_scorer.ts

crate::domain::events::PaymentEvent

Kind: Code library_first_party Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs

github.com/labeth/engineering-model-go/validate

Kind: Code library_first_party Owner: FU-CHECKOUT Source: checkout_api.go

fmt

Kind: Code library_stdlib Owner: FU-CHECKOUT Source: checkout_api.go

checkout_api.go

Kind: Code source_file Owner: FU-CHECKOUT Source: checkout_api.go

domain/events.rs

Kind: Code source_file Owner: FU-PAYMENT-AUTHORIZATION Source: domain/events.rs

payment_engine.rs

Kind: Code source_file Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs

risk_scorer.ts

Kind: Code source_file Owner: FU-RISK-SCORING Source: risk_scorer.ts

support/audit_envelope.ts

Kind: Code source_file Owner: FU-RISK-SCORING Source: support/audit_envelope.ts

CODE-AUTHORIZEPAYMENT

Kind: Code symbol Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs:10

CODE-BUILDREVIEWREASON

Kind: Code symbol Owner: FU-RISK-SCORING Source: risk_scorer.ts:49

CODE-CALCULATERISKSCORE

Kind: Code symbol Owner: FU-RISK-SCORING Source: risk_scorer.ts:7

CODE-CLASSIFYRISK

Kind: Code symbol Owner: FU-RISK-SCORING Source: risk_scorer.ts:20

CODE-CREATEAUDITRECORD

Kind: Code symbol Owner: FU-RISK-SCORING Source: risk_scorer.ts:36

CODE-ENRICHAUDITMETADATA

Kind: Code symbol Owner: FU-RISK-SCORING Source: risk_scorer.ts:44

CODE-HANDLEBANKLINKUNAVAILABLE

Kind: Code symbol Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs:33

CODE-HANDLEBANKUNAVAILABLE

Kind: Code symbol Owner: FU-CHECKOUT Source: checkout_api.go:31

CODE-ISHIGHRISK

Kind: Code symbol Owner: FU-RISK-SCORING Source: risk_scorer.ts:28

CODE-NOTIFYSUPPORT

Kind: Code symbol Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs:51

CODE-PERSISTAUDITRECORD

Kind: Code symbol Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs:59

CODE-PLACEINREVIEW

Kind: Code symbol Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs:43

CODE-REQUESTBANKAUTHORIZATION

Kind: Code symbol Owner: FU-PAYMENT-AUTHORIZATION Source: payment_engine.rs:25

CODE-SHOWDECLINEREASON

Kind: Code symbol Owner: FU-CHECKOUT Source: checkout_api.go:36

CODE-SHOWRETRYOPTION

Kind: Code symbol Owner: FU-CHECKOUT Source: checkout_api.go:41

CODE-SHOWTEMPORARYUNAVAILABLE

Kind: Code symbol Owner: FU-CHECKOUT Source: checkout_api.go:46

CODE-STARTSESSION

Kind: Code symbol Owner: FU-CHECKOUT Source: checkout_api.go:15

CODE-SUBMITAUTHORIZATIONREQUEST

Kind: Code symbol Owner: FU-CHECKOUT Source: checkout_api.go:26